

MsShellcodeDev-Rev1

Toolset Turning C Programmers to
Shellcode Developers

- Chinfeng Wu

Outline

- Self Intro
 - Who Am I?
 - MsShellcodeDev-Rev1 - Story
- Features of MsShellcodeDev-Rev1
 - Getting DLL Base
 - Getting API by Hash/Ord
 - Dealing with API Forwarding
- How to Use MsShellcodeDev-Rev1
 - Project Type to Use (Have Pure C Project Created)
 - C/C++ Features to Prevent (Don't Do THAT)
 - Comments to Add (Do THAT)
- Demonstration – Popup SysInfo Scene, Recon Scene, License Checking Scene
- Restrictions, Future Works & Conclusion
 - Restrictions: Based on MS IDE
 - Future Work: Shellcode as Function/Sub-routine Support
 - Conclusion

Who Am I?

- Degrees
 - Bachelor of Computer Science, National Defense University, TW, 2000
 - Master of Science in Computer Science, George Washington University, US, 2012
- Experiences
 - Ministry of National Defense, Taiwan, Programming Officer, 2000-2010
 - Ministry of National Defense, Taiwan, Cyber Security Specialist, 2012-2024
- Certificates
 - EnCE, 2016
 - ECIHv2, 2023



MsShellcodeDev-Rev1 - Story

- Course Taken, Fall 2010@GWU: CSCI 287 - Computer Network Defense
 - “Understanding Windows Shellcode” by Skape
(URL: <https://www.hick.org/code/skape/papers/win32-shellcode.pdf>)
- Reversing Poison Ivy RAT as Malware Analyzer
 - Original Shellcode Disassembled, Compatibility Problems on Different Win32 Platforms Figured Out
 - Poison Ivy RAT Patched for FUN (Works on All Windows Versions, **WILL NEVER RELEASE**, But Demonstration? Available @ URL: <https://github.com/SodaMtnDew/MsShellcodeDev/blob/main/videos/PatcheDPI232@Win11.mp4>)
- Course Taken: “Windows APT Warfare” @ HITCON 2019
 - “shellDev.py” Introduced by aaadress1 (Sheng-Hao Ma)
 - The Idea of MsShellcodeDev-Rev1 Emerged **THEN**

Features

- Getting DLL Base Address
 - Using DLL Name Hash to Find DLL Base from PEB (For Methods From Skape's PDF, Only PEB Works on ALL NT Based OS)
 - Using LoadLibraryA/LoadLibraryExA API & DLL Name to Load & Get Base of DLL NOT Initialized in PEB
- Getting API Address by Hash/Ordinal
 - Using DLL Base & Function Name Hash (From Skape's PDF, Better Change the Hash Function Used) or Ordinal to Get API Address
- Dealing with API Forwarding
 - Determine Whether Returned Address Point to API Address or Forwarded String
 - Recursive Calling for Both DLL Base & API Address

Features – DLL Base

- Skate's PEB Method of Getting Kernel32
 - Thanks to Geoff Chappell @ <https://www.geoffchappell.com/index.htm>

PEB x86 - FS:[0x30], x64 - GS:[0x60]

Offset (x86)	Offset (x64)	Definition	Versions
0x00	0x00	BOOLEAN InheritedAddressSpace;	all
0x01	0x01	BOOLEAN ReadImageFileExecOptions;	3.51 and higher
0x02	0x02	BOOLEAN BeingDebugged;	3.51 and higher
0x08	0x10	PVOID ImageBaseAddress;	all
0x0C	0x18	PEB_LDR_DATA *Ldr;	all

Get 2nd Entry in InInitializationOrderModuleList

Offset (x86)	Offset (x64)	Definition	Versions
0x00	0x00	ULONG Length;	all
0x04	0x04	BOOLEAN Initialized;	all
0x08	0x08	PVOID SsHandle;	all
0x0C	0x10	LIST_ENTRY InLoadOrderModuleList;	all
0x14	0x20	LIST_ENTRY InMemoryOrderModuleList;	all
0x1C	0x30	LIST_ENTRY InInitializationOrderModuleList;	all

- KernelBase.dll Introduced
 - Vista & Prior, 2nd is Kernel32; 7 & Later, 2nd is KernelBase, 3rd is Kernel32
 - LoadLibraryA in Kernel32, NOT in KernelBase of Windows 7
 - LoadLibraryExA in Both DLLs, Get this API from 2nd Module to Get Kernel32
 - Poison Ivy Patched Using this Trick

Features – DLL Base (Cont.)

Why Change DLL Base Getting Mechanism?

- Per-process Reference Count of DLL
 - Count Increase When LoadLibrary/LoadLibraryEx Called; Decrease When FreeLibrary Called; DLL Fully Unloaded When **Count=0**
 - Could Be Used as Signature by EDR/AntiVirus to Detect Malware
 - DLL Could be Loaded Without Calling API
 - When a Given DLL Loaded, THOSE Imported DLLs WILL Be Loaded, Too
 - Best Practices:
 - Using Hash of DLL Filename to Enumerate LIST_ENTRY in PEB_LDR_DATA to Locate Loaded Library
 - No LIST_ENTRY Found → Call LoadLibrary/LoadLibraryEx
- ※Ntdll, Kernel32 & KernelBase Loaded into PEB When Initializing, Just Use Hash (Without Giving Name) to Get Base Address

Features – API by Hash/Ord

- Skate's Hash Function Used to Get API Address
 - "shellDev.py" by aaaddress1 Use Same "add Char" THEN "ror Hash, 0x0d" Algorithm, too. Coincident?
 - Some Hash Values are Already Signature in Antivirus Software
 - Poison Ivy Used a More Complicated Hash Function
- Customized Hash Function of MsShellcodeDev-Rev1
 - Keep in Mind when Developing: Same Hash Function Be Used for Both API & DLL
 - API: Case Sensitive, ANSI String (No Modification Needed)
 - DLL: Case Insensitive (Make ALL Alphabets Upper or Lower case), Both ANSI & UNICODE String
- Get API Address by Function Ordinal
 - Previously Undocumented API Exported in Ordinal; For Backward Compatibility, Those Ordinals NEVER Changed
 - Support Getting API by Ordinal = Higher Compatibility
 - Best Sample APIs: SHCreateMemStream & IStream_* in shlwapi.dll
 - Know Those APIs from MS Documents, Or An Option Much Better – ["Geoff Chappell's Home"](#)

```
compute_hash:
    xor     edi, edi
    xor     eax, eax
    cld
compute_hash_again:
    lodsb
    test    al, al
    jz      compute_hash_finished
    ror     edi, 0xd
    add     edi, eax
    jmp     compute_hash_again
compute_hash_finished:
```

Features – API Forwarding

- API Forwarding Exists Everywhere
 - “HeapAlloc” of Kernel32 Constantly Forwarded to NTDLL.RtlAllocateHeap, Since Win2k

→ **HeapAlloc <forwarded to NTDLL.RtlAllocateHeap>**

- Most Solutions Use “GetProcAddress” API to Deal with That
 - Some API String Might Be Signature to EDR/Antivirus, Need Obfuscation
 - Using “GetProcAddress” Could Be Categorized as Malicious Behavior for Some EDR/Antivirus
- Some Figured Out, But Not Solved
 - If API RVA in Export Table, It’s API Forwarding
 - In the GitHub Repository Release Recently, API Forwarding = Return NULL Address?
- Solution in MsShellcodeDev
 - Forwarding String Format:
[DLL].[API] or [DLL].#[Ordinal]
 - Use Features 1, 2 to Get DLL Base & API Address

```
DWORD dwFunctionRVA = pdwFunctions[pwOrdinals[i]];
if (dwFunctionRVA >= exportDir.VirtualAddress &&
    dwFunctionRVA < exportDir.VirtualAddress + exportDir.Size) {
    return NULL;
}
```

8

How to Use – VS IDE Projects

- Recommended Project Type
 - Win32 Project, No Console
 - In Fact, Project Types Don't Really Matter, Just Keep in Mind that "PowerShell/Terminal" NOT Preferred Since on Windows Platform, "Shell" = "Additional Process"
- Optimization?
 - For Vcxproj to Be Converted: No Need, Using Config of ShellcodePrototype
 - For ShellcodePrototype: No Optimization
 - Considering Compatibility & Stability
 - Feel Free to Conduct Your Own Experiment on ShellcodePrototype
 - Reminder: Instruction Set Extensions NOT Preferred for Compatibility
 - Reminder: Prevent Using SIMD Instruction Since in that CASE, Raw Data Will Be Stored in ".rdata" Section thus Made Codes NOT "Position Independent"

How to Use – Don't Do THAT

- Features to Prevent
 - Global Variables (Will be Stored in “.data” Section)
 - C++ Features, Including Classes (MFC, ATL), Interfaces, etc.
 - CRT Functions (Security-Enhanced Versions)
 - Any Feature that Let Compiler Generate Position Dependent Code or Put Data to Other Sections (For Example: switch/case statement, SIMD, etc.)
 - When Initialize Array/Struct as Local Variables, Provide **ALL** Elements, or `_memset` (from MSVCRT) WILL be Used to Set the Rest & Cause Error
- Exceptions
 - Classes, Interfaces that Could Use Flat Win32 API (Without Wrapper) to Access ARE OK! (MemStream & GdiPlus, etc.)
 - CRT Functions From System Provided DLL “msvcrt” - Better Check ExportTable of msvcrt.dll on Windows 2000 Before Using CRT APIs (DDK/WDK Developers SHOULD BE Familiar with That)

How to Use – Do THAT

- Add Comments After Debugged
 - Classical C Comment Format Used, However..
 - Using “/*/” as Both Head & Tail for Comments Before API Call, Put “[DLL]” or “[DLL].#[Ordinal]” in the Middle
 - Only the 1st Appearance of API in Function Blocks SHOULD be Commented
 - Leave Constant Strings Alone:
 - In Normal Cases, Compilers put Constant Strings in “.rdata” Section
 - “VcxprojToShellcodeCompiler” Converts All Constant Strings Into Local DWORD Array Variables
- Execute “VcxprojToShellcodeCompiler” on Well Prepared Vcxproj
 - If Error, Run “VcxBuild.bat” in “PicProj” Folder to Debug
- Test Shellcode After Extracted
 - Use the “Right” Loader & Shellcode “Pairs”
 - Keep Compatibilities in Mind

Demonstration One

- Built-in ShellcodeSample (GetSysInfo)
 - Key Features: Using “IStream” Interface in Pure C & Win32 APIs to Deal with Dynamic Sized Buffers
 - Available @ URL:
<https://github.com/SodaMtnDew/MsShellcodeDev/blob/main/videos/GetSysInfo@Windows.mp4>

Demonstration Two

- Reconnaissance Scene (GetShellIcon Remotely)
 - Key Features: Using GDI+ in Flat API & Retrieve Default Icon Images of Specific File Formats
 - Available @ URL:
<https://github.com/SodaMtnDew/MsShellcodeDev/blob/main/videos/ReconAgents2ReconCommand.mp4>

Demonstration Three

- Non-Malicious Scene - License Check
 - Key Features: iSerialNumber Exists to Make Thumb drives as Dongles; Using HMAC, Key to Obfuscate License Info in Shellcode
 - Available @ URL:
<https://github.com/SodaMtnDew/MsShellcodeDev/blob/main/videos/CheckLicense@USB.mp4>

Future Works

- Core Function Integrated & Stored as RVA
 - Could Help Make Modularized RAT Such as Poison Ivy
 - How to Detect & Deal With Functions Using Win API?
 - Could Make Anti-Memory Scan Malware Such as Waterbear
 - How to Control Shellcode Buffers? Which to Keep? Which to Erase?
- Any Suggestion For Even Further Revision to Be?

Restrictions

- Restriction Based on Platform Versions
 - Windows Shellcode = Windows Platform Only
 - No Itanium/ARM32 Platform Tested Since No Host/Guest OS Could be Used; RISC-V Possible if Windows Support IT in the Future
- Restriction Based on MS IDE Versions
 - No Compiler for Specific CPU Arch = No Shellcode Possible
 - MsShellcodeDev-Rev1 Officially Supports VS 2019 & Newer Version MS IDE
- Restriction Based on Other Factors
 - Win32 Shellcode OK For All Windows, Use for Target Recon
 - None-Native CPU OpCodes Executed by Emulator/Simulator/Virtualization On Windows 64 bits (amd64, arm64) Version – Hook Based Detection Worked on IT! To be FUD, x64/arm64 Shellcode.
 - Best Developing IDE - WoA: ALL of x86/x64/arm64 Shellcode Work!

Conclusion

- Windows Syscall Worked, Not Recommended. Why?
 - NTDLL Provides “Pure” Windows Syscall, Most “Undocumented”
 - NTDLL “NOT Having Socket” API Included
 - Directly Using Windows Syscall (i.e. Calling API Directly from NTDLL.dll, Either By Ordinals or Names) Might Be “Signature” of Shellcode-based “Malware Detection”
- Old Tricks Worked, Works & Will Keep Working
 - Unless Microsoft “GET RID OF BACKWARD COMPATIBILITY”
 - However, Some Tricks Introduced for Security Reason, Such as GS (Security Cookie), ASLR, SafeSEH & DEP – Finding Countermeasures (as ROP to DEP for Example) Better Than Finding Detour

Thanks for Coming to my Debut